

DataSentry - Technical Specification

Version 1.0 | Target: .NET 8, WPF, Windows

1. Overview

Purpose: A local-only Windows desktop application that recursively scans a user-selected directory, detects text patterns matching personal/confidential/GDPR-relevant data categories, and produces a risk-scored report per file. No network calls, no telemetry, no data leaves the machine.

Non-goals (explicitly out of scope for v1): cloud sync, OCR of scanned images, ML/NER-based name detection, real-time file-system watching, multi-user/server mode, editing or redacting the source files themselves.

2. Functional Requirements

FR1 - Folder selection

- User selects a root directory via a native folder picker, including the option to select an entire drive root (e.g. `C:\`)
- App recursively enumerates all files under it, respecting an extension allowlist and a max-file-size cutoff (default 50 MB, configurable)
- Symbolic links and junctions are not followed (avoid infinite loops / scanning outside intended scope)
- **Default exclusion list**, applied automatically and shown to the user before a drive-root scan starts (overridable in settings, see §13):
 - `Windows`, `Program Files`, `Program Files (x86)`, `ProgramData`
 - `$Recycle.Bin`, `System Volume Information`
 - The app's own install/output/log directories
 - Common dev caches: `node_modules`, `.git`, `bin`, `obj`, `packages`, `.nuget`

FR2 - Detector categories

User can enable/disable detector categories before scanning. Default: all enabled.

Category	Detectors	Validation
Identity	PESEL	regex + weighted checksum
Financial	IBAN, Credit Card	regex + mod-97 (IBAN) / Luhn (card)
Contact	Email, Phone (PL/EU)	regex
Network	IPv4, IPv6	regex
Keyword	PL/EN sensitive terms	case-insensitive term list

FR3 - File text extraction

Extension	Method
<code>.txt</code> , <code>.csv</code> , <code>.json</code> , <code>.log</code> , <code>.md</code> , <code>.xml</code>	direct UTF-8/ANSI read with encoding detection
<code>.docx</code>	DocumentFormat.OpenXml
<code>.xlsx</code>	DocumentFormat.OpenXml (all sheets, cell text only)
<code>.pdf</code>	PdfPig, text layer only (no OCR in v1)
anything else	skipped, logged as "unsupported" in results, not an error

Files that fail extraction (corrupt, password-protected) are recorded with `Error` populated and `Risk = None`, not silently dropped.

FR4 - Scanning execution

- Scan runs on background threads, UI remains responsive throughout
- Live progress: files scanned / total, current file path, matches found so far, throughput (files/sec)
- Cancellable mid-scan; partial results already computed remain visible
- Configurable parallelism (default: `Environment.ProcessorCount`)

FR5 - Risk scoring

Per file, aggregate all matches into a single `RiskLevel`:

Condition	Risk
≥1 match in Identity or Financial category	High
≥1 match in Contact or Network category, no Identity/Financial	Medium
Only Keyword matches	Low
No matches	None

Exact thresholds configurable in `RiskScorer` (documented as a pure function for easy testing/tuning).

FR6 - Results display

- Sortable/filterable grid: file path, risk badge (color-coded), match count, file size, last modified
- Selecting a row shows a detail pane: list of matches with detector name, category, position in file, and a **redacted** snippet (never the raw value)
- Filter by risk level and/or category

FR7 - Reporting

- Export to HTML (styled, summary + detail tables) and CSV (flat, one row per match)
- Report includes: scan root, timestamp, options used (which detectors enabled), summary counts by risk level, full detail table

FR8 - Replay/re-scan

- Re-run a scan on the same root without reselecting the folder
- Option to scan only files modified since the last scan (mtime-based)

3. Non-Functional Requirements

Requirement	Target
Privacy	No network calls anywhere in the app. No matched raw values persisted to disk in logs or app state - redacted snippets only
Performance	≥200 files/sec on plain text files on a typical dev laptop (SSD, 8 cores); large files (>10MB) streamed rather than loaded fully where the extractor supports it
Responsiveness	UI thread never blocked >100ms during a scan, even at hundreds of thousands of files

Requirement	Target
Reliability	A single file's extraction failure, permission denial, or path error never aborts the overall scan
Memory	Bounded regardless of scan size - results streamed to the UI incrementally, not buffered entirely in memory before display (see §13)
Portability	Self-contained single-file publish option (<code>dotnet publish -r win-x64 --self-contained</code>) so it runs without a pre-installed .NET runtime
Accessibility	Keyboard navigable grid, standard WPF accessibility (<code>AutomationProperties</code>) on key controls

4. Data Model

csharp

```

public interface IDetector
{
    string Name { get; }
    string Category { get; }
    IEnumerable<DetectionMatch> Scan(string content);
}

public record DetectionMatch(
    string DetectorName,
    string Category,
    int Position,
    string RedactedSnippet,
    double Confidence);

public record FileScanResult(
    string FilePath,
    long FileSizeBytes,
    DateTime LastModifiedUtc,
    IReadOnlyList<DetectionMatch> Matches,
    RiskLevel Risk,
    TimeSpan ScanDuration,
    string? Error);

public enum RiskLevel { None, Low, Medium, High }

public record ScanOptions(
    IReadOnlySet<string> EnabledCategories,
    long MaxFileSizeBytes,
    int MaxDegreeOfParallelism,
    bool IncrementalSinceLastScan);

public record ScanProgress(
    int FilesScanned,
    int TotalFiles,
    string CurrentFile,
    int MatchesFound,
    double FilesPerSecond);

```

5. Architecture

DataSentry.Core	- detectors, models, scan engine, risk scoring. No IO/UI deps.
DataSentry.Extraction	- text extraction per file format. Depends on Core.
DataSentry.App	- WPF, MVVM. Depends on Core + Extraction. No business

logic in code-behind.

DataSentry.Cli - optional headless runner, same Core+Extraction, for scripting/demo.

Key architectural rule: `Core` has zero dependency on file IO or any format-parsing library - it operates purely on `string content` passed in by the `Extraction` layer. This keeps detectors trivially unit-testable and reusable outside the desktop app (e.g. as a future NuGet package or CLI).

Scan pipeline (single file):

```
FileSystemWalker → ExtractorFactory.GetExtractor(path) →  
ITextExtractor.ExtractTextAsync  
    → for each enabled IDetector: Scan(content) → matches  
    → RiskScorer.Score(matches) → FileScanResult
```

Concurrency model:

- `Directory.EnumerateFiles` (lazy, streaming) feeds a `System.Threading.Channels.Channel<string>`
- `Parallel.ForEachAsync` with `ScanOptions.MaxDegreeOfParallelism` consumes the channel, running extraction + detection per file
- Results are yielded back via `IAsyncEnumerable<FileScanResult>`, consumed by the ViewModel and marshalled to the UI thread via `Dispatcher` for grid updates
- `IProgress<ScanProgress>` reports throughput every N files or every 250ms, whichever is less frequent, to avoid UI-thread flooding

6. UI Specification (MVVM)

MainWindow layout:

```

| [Folder: .....] [Browse] [Start Scan] [Cancel] |
| Progress: [=====] 340/1200 files 180 f/s |
|-----|
| Categories | File | Risk | Matches | Size | Mod. |
| [x] Identity | report.docx | HIGH | 3 | 2.1MB | ... |
| [x] Financial | notes.txt | LOW | 1 | 4KB | ... |
| [x] Contact | data.csv | MED | 5 | 890KB | ... |
| [x] Network | ... | | | | |
| [x] Keyword | | | | | |
|-----|
| Detail: report.docx |
| • PESEL at pos 1204 - "48*****12" (Identity, 0.95) |
| • Email at pos 3310 - "j***@***.com" (Contact, 0.90) |
|-----|
| [Export HTML] [Export CSV] [Open File Location] |
|-----|

```

ViewModel contract:

csharp

```

public partial class MainViewModel : ObservableObject
{
    [ObservableProperty] private string? selectedFolder;
    [ObservableProperty] private ScanProgress? progress;
    [ObservableProperty] private bool isScanning;
    [ObservableProperty] private FileScanResult? selectedResult;

    public ObservableCollection<FileScanResult> Results { get; } = new();
    public ObservableCollection<DetectorCategoryOption> Categories { get; } = new();

    [RelayCommand] private async Task StartScanAsync();
    [RelayCommand] private void CancelScan();
    [RelayCommand] private async Task ExportHtmlReportAsync();
    [RelayCommand] private async Task ExportCsvReportAsync();
    [RelayCommand] private void OpenFileLocation(FileScanResult result);
}

```

Risk badges color-coded: None=gray, Low=yellow, Medium=orange, High=red - via a

`RiskLevel` → `Brush` `IValueConverter`.

7. Detector Specifications

PESEL (Polish national ID)

- Pattern: `\b\d{11}\b`
- Validation: weighted checksum, weights `[1,3,7,9,1,3,7,9,1,3]` mod 10 against 11th digit
- Also validate embedded birthdate is plausible (month/century encoding) to cut false positives on generic 11-digit numbers

IBAN

- Pattern: `\b[A-Z]{2}\d{2}[A-Z0-9]{11,30}\b`
- Validation: mod-97 checksum per ISO 13616 (rearrange, convert letters to numbers, mod 97 == 1)

Credit card

- Pattern: `\b(?:\d[ -]*?){13,19}\b`
- Validation: Luhn algorithm
- Exclude obvious false positives (sequential digits, all-same-digit)

Email

- Standard RFC-5322-lite pattern, no full spec compliance needed for a detector

Phone (PL/EU)

- Patterns for `+48 XXX XXX XXX`, `XX-XXX-XX-XX`, generic `+CC` prefixed E.164-ish formats

IP address

- IPv4: standard dotted-quad regex with 0-255 range validation per octet
- IPv6: standard colon-hex pattern

Keyword

- Case-insensitive substring/word-boundary match against a configurable term list (PL + EN), loaded from an embedded resource so it's easy to extend without code changes
-

8. Testing Specification

Layer	Approach
Detectors	<code>[Theory]</code> / <code>[InlineData]</code> per detector: ≥ 3 known-good matches, ≥ 3 known-bad near-misses, checksum edge cases (valid vs. invalid PESEL/IBAN/card)
RiskScorer	Table-driven tests covering every category/count combination in FR5
Extractors	Fixture files (<code>tests/fixtures/sample.docx</code> etc.) with known expected text, asserted via extraction
ScanEngine	Integration test against a temp directory with a mixed fixture set, asserting result count, risk distribution, and that cancellation stops mid-scan cleanly
UI	Manual test checklist (see §9) rather than automated UI tests for v1 - not worth the investment at this scope

9. Manual Test Checklist (pre-release)

- ☐ Scan a folder with 0 files - no crash, "no files found" state
- ☐ Scan a folder with 10,000+ small files - UI stays responsive, progress updates smoothly
- ☐ Scan includes a password-protected .docx - recorded as error, scan continues
- ☐ Cancel mid-scan - partial results remain, no orphaned background threads
- ☐ Disable all categories, scan - all files show Risk = None
- ☐ Export HTML report - opens correctly in a browser, summary counts match grid
- ☐ Export CSV - opens correctly in Excel, one row per match
- ☐ Re-scan with "incremental since last scan" - unchanged files skipped
- ☐ Select `C:\` as scan root - confirmation dialog appears before scan starts
- ☐ Full-drive scan hits `C:\Windows\CSC` or similar access-denied folder - scan continues, folder logged as skipped, no crash
- ☐ Full-drive scan encounters a path >260 chars - handled gracefully, not fatal
- ☐ Full-drive scan encounters a locked system file (e.g. `pagefile.sys`) - recorded as error, scan continues
- ☐ Pause mid-scan, then resume - enumeration progress and existing results are retained
- ☐ Cancel a large in-progress scan - responds within a few seconds, not stuck waiting for the whole tree to finish enumerating

10. Configuration & Settings

Stored in `%AppData%/DataSentry/settings.json`:

```
json
{
  "maxFileSizeBytes": 52428800,
  "maxDegreeOfParallelism": 8,
  "enabledCategories": ["Identity", "Financial", "Contact", "Network", "Keyword"],
  "excludedExtensions": [".exe", ".dll", ".zip"]
}
```

No sensitive data (matches, file paths from past scans) persisted between sessions in v1 - each run starts clean, by design, to minimize the app's own data footprint.

11. Full-Drive Scan Robustness

Scanning `C:\` is a fundamentally different workload than scanning a project folder - potentially millions of files, deep system trees, locked files, and permission boundaries. This section makes that scenario a first-class supported case, not an edge case that happens to work.

11.1 Enumeration must never crash on a single bad directory

`Directory.EnumerateFiles(..., SearchOption.AllDirectories)` throws and **aborts the entire enumeration** on the first `UnauthorizedAccessException` or `PathTooLongException` it hits - guaranteed to happen on a full-drive scan (e.g. `C:\Windows\CSC`, `C:\Users\*\AppData` reparse points). The walker must **not** use the built-in recursive overload directly. Instead, walk manually and isolate failures per directory:

```
csharp
```

```

public static IEnumerable<string> SafeEnumerateFiles(string root, IReadOnlySet<string>
{
    var stack = new Stack<string>();
    stack.Push(root);
    while (stack.Count > 0)
    {
        var dir = stack.Pop();
        if (IsExcluded(dir, excludedDirs)) continue;

        IEnumerable<string> subDirs = Array.Empty<string>();
        IEnumerable<string> files = Array.Empty<string>();
        try
        {
            subDirs = Directory.EnumerateDirectories(dir);
            files = Directory.EnumerateFiles(dir);
        }
        catch (UnauthorizedAccessException) { continue; }
        catch (PathTooLongException) { continue; }
        catch (DirectoryNotFoundException) { continue; } // race: deleted mid-scan
        catch (IOException) { continue; } // e.g. device not ready

        foreach (var f in files) yield return f;
        foreach (var d in subDirs) stack.Push(d);
    }
}

```

One denied folder skips that folder only, logged to a "skipped paths" list shown at the end of the scan - never surfaced as a scan-ending error.

11.2 Long path support

Windows' legacy 260-char `MAX_PATH` limit is hit often in deep `AppData`/`node_modules` trees on a full-drive scan. Add to `DataSentry.App.csproj`:

```

xml
<PropertyGroup>
  <LongPathsEnabled>true</LongPathsEnabled>
</PropertyGroup>

```

plus the corresponding `app.manifest` `longPathAware` entry. Paths that still fail are caught per §11.1, not fatal.

11.3 Locked / in-use files

System files (page file, hibernation file, registry hives, files locked by running processes) throw `IOException` on open. Treat identically to extraction failures already specified in

FR3: record `FileScanResult` with `Error` set, `Risk = None`, continue. Do not retry with elevated sharing modes - this app has no business forcing its way into locked system files.

11.4 Progress reporting at unknown scale

Counting total files up front (a full pre-walk) is itself expensive at drive scale and delays the user seeing any progress. Use a two-phase model:

- Phase 1 (first few seconds): indeterminate progress bar, "Discovering files... (found so far: X)"
- Phase 2: once enumeration is comfortably ahead of processing (buffered via the channel), switch to a determinate bar using the running discovered-count as an estimated denominator, labeled "~" since it can still grow
- Enumeration and processing run concurrently throughout - never block scan start on enumeration finishing first, consistent with the channel-based pipeline in §5

11.5 Memory bounds

- Results are pushed to the UI incrementally per file, not accumulated in a background list first
- Raw extracted file text is never retained after detection runs on it - only `DetectionMatch` (with redacted snippets) survives past that file's processing
- Channel capacity is bounded (e.g. `BoundedChannelOptions(10_000)`) so a fast enumerator can't outpace slow consumers and balloon memory - producer awaits capacity via backpressure

11.6 Time expectations, pause/cancel

A full `C:\` scan can reasonably take minutes to over an hour depending on file count/disk speed.

- Cancellation is checked per-file (not per-directory) via `CancellationToken` threaded through `Parallel.ForEachAsync`, so Cancel responds within roughly one file's processing time even deep into a scan
- Add a **Pause** control (distinct from Cancel) that suspends the consumer side of the channel without losing enumeration progress or already-collected results - useful for stepping away mid-scan
- No auto-resume across app restarts in v1 - a crash or forced close loses in-progress state, consistent with the no-persistence privacy stance in §10

11.7 Confirmation before a drive-root scan

When the selected root is a drive root (or looks like one - e.g. contains `Windows` and `Users` as immediate children), show a confirmation dialog before starting:

"You've selected an entire drive. This may scan hundreds of thousands of files and take significant time. System and program folders are excluded by default - review or change this in Settings. Continue?"

Prevents an accidental multi-hour scan from a misclick.

11.8 Exclusions are visible and editable, not hidden

The default exclusion list (FR1) is shown in a Settings screen with add/remove controls, not silently applied - a privacy-focused tool should never look like it's hiding its own scan scope from a technically curious user.

12. Distribution

- `dotnet publish src/DataSentry.App -c Release -r win-x64 --self-contained -o publish`
 - Attached as a GitHub Actions build artifact (see build plan doc) and optionally as a GitHub Release asset
 - No installer required for v1 - a runnable `.exe` is sufficient for a portfolio demo
-

13. Roadmap (post-v1, not in scope now)

- OCR for scanned/image-only PDFs (Tesseract)
- ML.NET-based name entity recognition to catch personal names, not just structured identifiers
- Browser-extension companion for real-time warnings on PII in web forms/AI chat inputs (opt-in, client-side only - see earlier discussion on why this must stay non-covert)
- Scheduled/recurring scans with a system tray presence